Student "Notes Exchange" Portal

---

1. Introduction

A **Student Notes Exchange Portal** is a web-based platform designed to centralize and organize academic study materials for students.

In many colleges, study resources such as notes, previous year question papers, and guides are scattered across platforms like Google Drive and WhatsApp groups. This makes it difficult for students to find relevant materials, especially during exams.

This platform aims to provide a **centralized repository** where students can:

- Upload and share notes
- Search materials easily
- Access organized content by subject, semester, and branch

The project demonstrates concepts such as **file management systems, content ranking (voting), search functionality, and full-stack web development**.

---

2. Description

The Student Notes Exchange Portal allows students to upload, browse, and interact with study materials.

The application enables users to:

- Upload notes in PDF or image format
- Browse notes categorized by:
    - Semester
    - Subject
    - Branch
- Search notes using subject codes
- Upvote or downvote notes based on quality

The system provides:

- A structured and organized repository
- A voting system to highlight useful content
- Easy access to academic resources

This platform improves collaboration among students and ensures better access to quality study materials.

---

3. Problem Statement

Study materials are often scattered across multiple platforms like Google Drive and WhatsApp, which leads to:

- Difficulty in finding relevant notes
- Lack of organization
- Wasted time during exams

Students face challenges such as:

- No centralized platform for notes
- Difficulty in identifying high-quality materials
- Inefficient search systems

The challenge is to build a platform that:

- Organizes documents by semester, subject, and branch
- Allows file uploads (PDF/Image)
- Implements a voting system for content quality
- Provides a fast and efficient search feature

---

4. Scenario

Consider a student named **Rahul** preparing for exams.

- Rahul opens the Notes Exchange Portal
- Selects his **semester and subject**
- Uses the search bar to find notes by subject code
- Views a list of notes sorted by upvotes

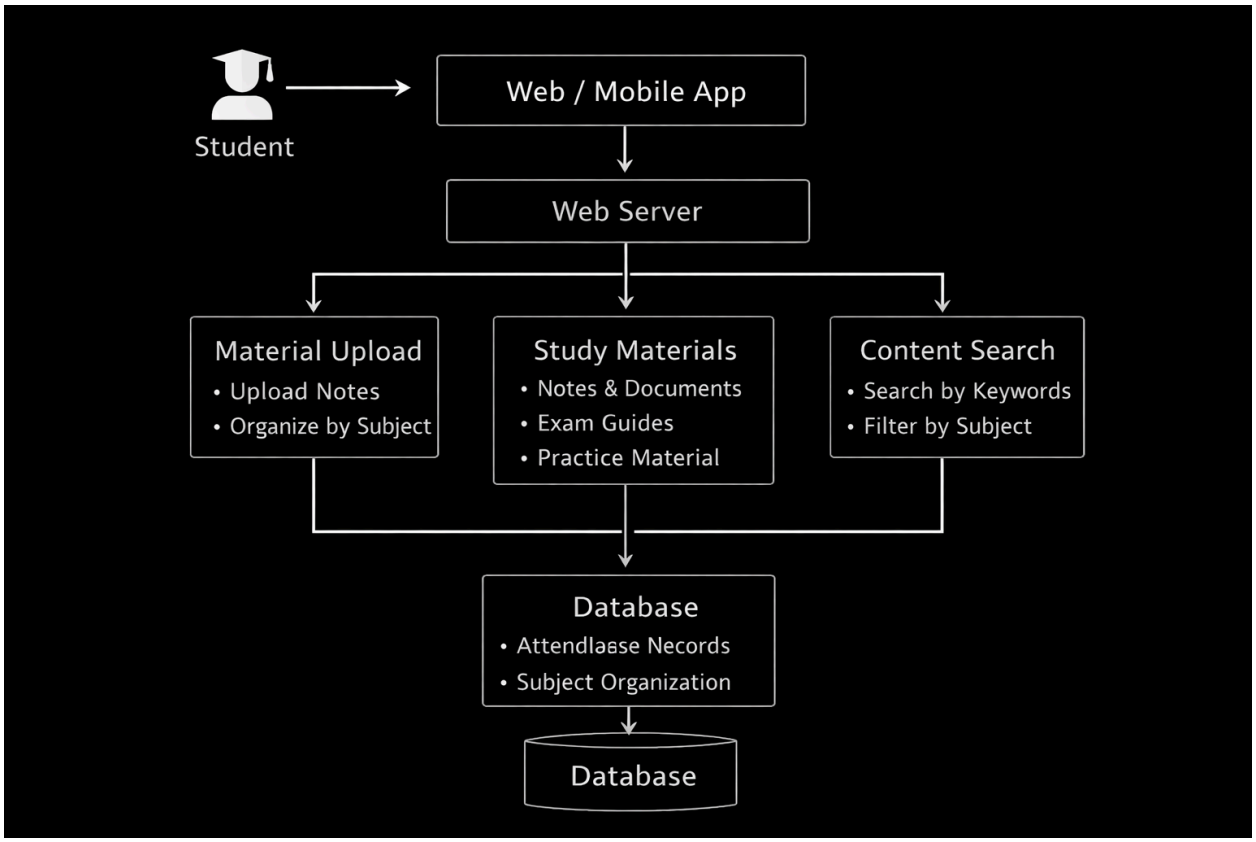
He:

- Downloads high-rated notes
- Uploads his own notes to help others
- Upvotes useful materials

This creates a collaborative and efficient learning environment.

---

5. Architecture



The system follows a **three-tier architecture**.

Frontend Layer

- Notes listing page
- Upload form (PDF/Image)
- Search bar
- Voting buttons (upvote/downvote)

Developed using:

- React.js
- Tailwind CSS

Backend Layer

- Handles file uploads
- Manages notes metadata
- Processes voting system
- Provides search APIs
- Authentication & authorization

Developed using:

- Node.js
- Express.js

Database Layer

- Stores users, notes, votes, and metadata

Implemented using:

- MongoDB

Communication Flow

Frontend (React)
↓ API Requests
Backend (Node.js + Express)
↓ Database Queries
MongoDB Database

---

6. Project Flow (System Flow)

Step 1: User Registration

User creates an account

Step 2: Authentication

User logs in

Step 3: Upload Notes

User uploads PDF/Image with metadata

Step 4: Storage

File is stored and metadata saved in database

Step 5: Browse Notes

Users explore notes by category

Step 6: Search

User searches notes by subject code

Step 7: Voting

Users upvote/downvote notes

---

7. User Flow

Registration

User → Sign Up → Login

Upload Flow

User → Upload Notes → Add Details → Submit

Browse Flow

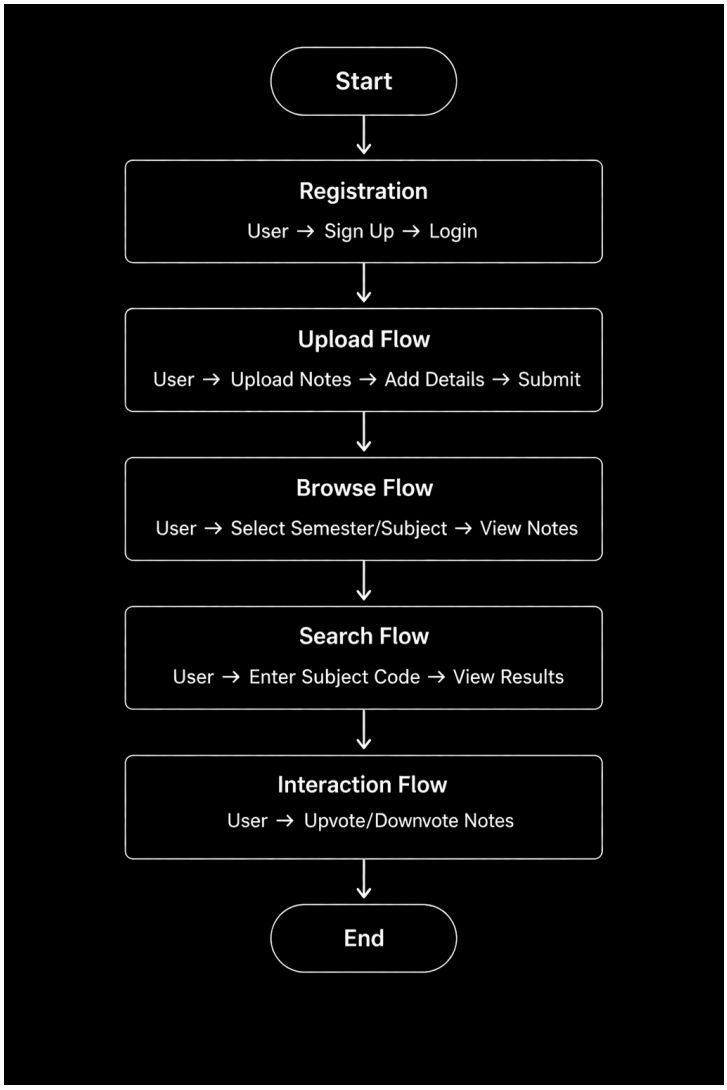
User → Select Semester/Subject → View Notes

Search Flow

User → Enter Subject Code → View Results

Interaction Flow

User → Upvote/Downvote Notes



8. Pre-Requisites

Frontend (React)

- React Components
- React Hooks
- React Router
- Axios
- TailwindCSS

Backend (Node.js & Express)

- Node.js fundamentals
- Express routing
- REST APIs
- Middleware
- JWT Authentication
- File upload handling (Multer)

Database (MongoDB)

- MongoDB collections
- CRUD operations
- Mongoose ODM
- Schema design

---

9. Required Technologies

Frontend

- React.js
- TailwindCSS

Backend

- Node.js
- Express.js
- JWT Authentication
- Multer (file upload)

Database

- MongoDB
- Mongoose

Development Tools

- VS Code
- Git
- GitHub
- Postman / Thunder Client

---

10. Suggested Database Collections

Users

- _id
- name
- email
- password
- createdAt

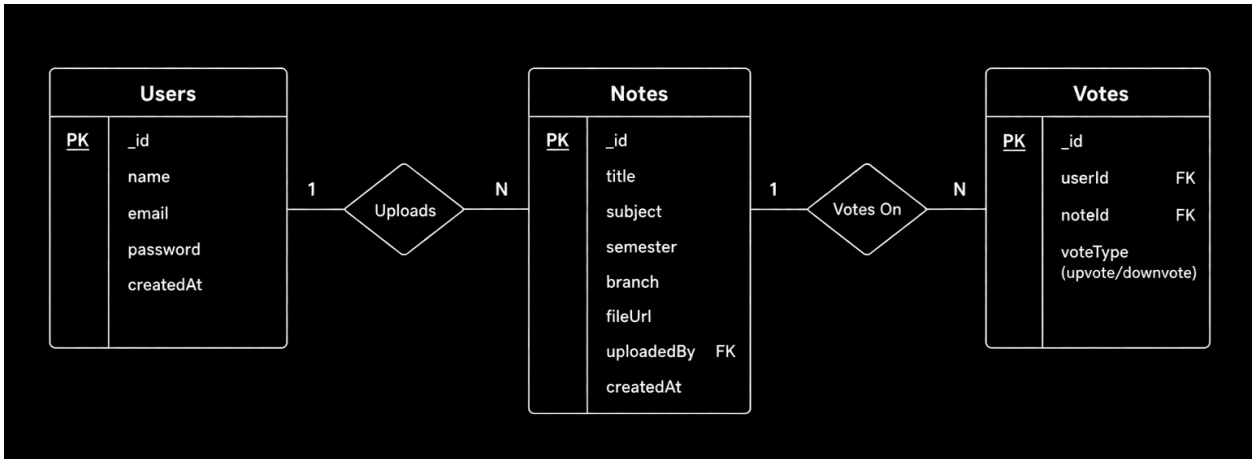
Notes

- _id
- title
- subject
- semester
- branch
- fileUrl
- uploadedBy (userId)
- createdAt

Votes

- _id
- userId
- noteId
- voteType (upvote/downvote)

ER Diagram



11. Key Features

Authentication

- Signup
- Login
- JWT authentication

Notes Management

- Upload notes
- View notes
- Categorization

Search System

- Search by subject code

Voting System

- Upvote notes
- Downvote notes

File Handling

- Upload PDF/Image
- Store and retrieve files

---

12. Optional Advanced Features

- Comments on notes
- Bookmark/favorites system
- File preview (PDF viewer)
- AI-based note recommendations
- Download tracking
- Admin moderation panel
- Cloud storage integration (AWS S3 / Cloudinary)

13. Learning Outcomes

By completing this project, developers will learn:

- Full-stack MERN development
- File upload and storage systems
- Search functionality implementation
- Voting/ranking systems
- Database schema design
- User-generated content management
- API design and integration